Laboratory No. 6: CNN Architectures

ResNet, ResNeXt, and Transfer Learning

Lab 6: CNN Architectures

Faculty of Mathematics and Information Science
Warsaw University of Technology

March 30, 2026

Laboratory Rules

- All places you need to implement something are highlighted
- To check every part: run `python ./part_1.py`
- `helpers.py` is not important - only read function descriptions if needed
- On this Lab please don't use chat

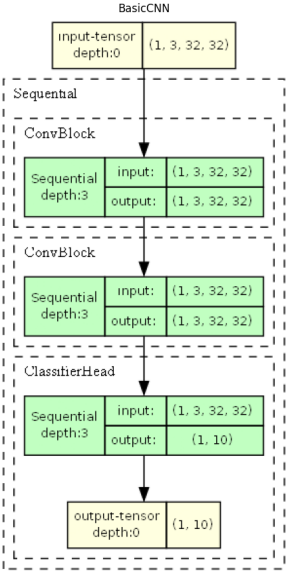
What You Will Do

1. **Part 1:** Implement BasicCNN and ResNet
2. **Part 2:** Implement ResNeXt architectures
3. **Part 3:** Implement Transfer learning

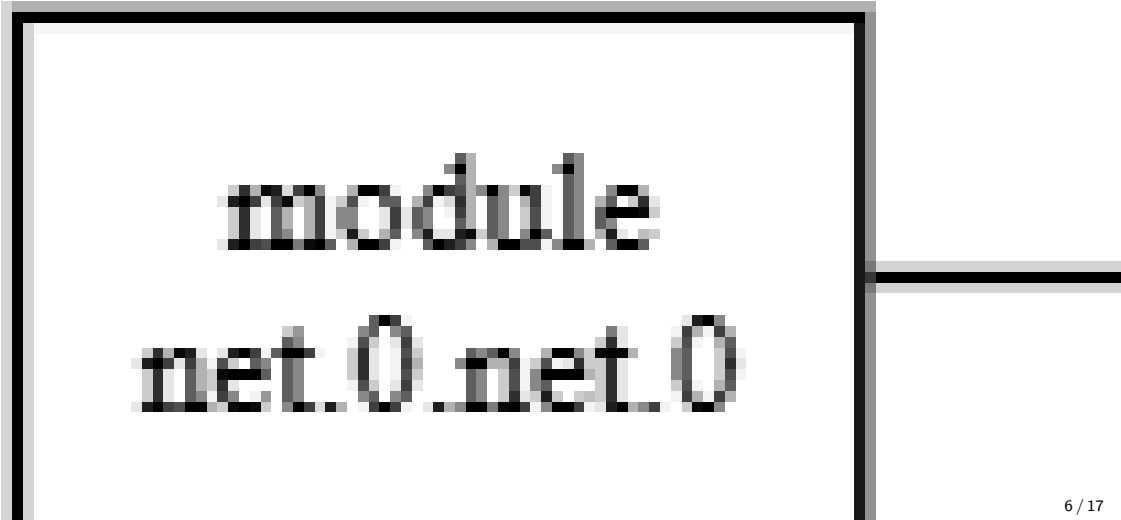
Part 1: Basic CNN vs ResNet

- **ConvBlock:** Building block with 2 Conv2d layers + ReLU (optional BatchNorm)
- **ClassifierHead:** AdaptiveAvgPool2d + Flatten + Linear
- **BasicCNN:** ConvBlock + ConvBlock + ClassifierHead
- **ResidualBlock:** Implements skip connection: $y = x + \text{ConvBlock}(x)$
- **SimpleResNet:** ConvBlock + 2 ResidualBlocks + ClassifierHead
- Compare training dynamics between plain CNN and ResNet

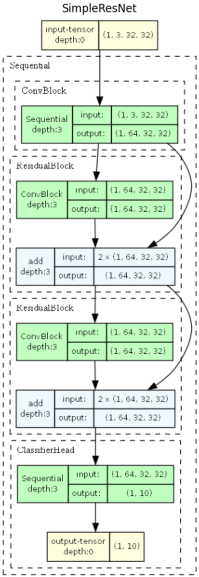
Part 1: BasicCNN Architecture



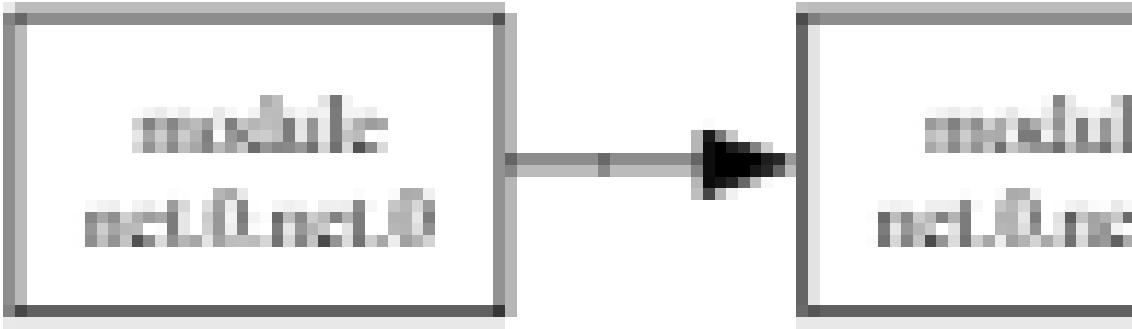
Part 1: BasicCNN Architecture



Part 1: SimpleResNet Architecture



Part 1: SimpleResNet Architecture



Part 1: Help Info

1. **nn.Module** - Base class for all neural network modules. Inherit from it and define `__init__` and `forward`
2. **AdaptiveAvgPool2d(1)** - Pools input to 1x1 spatial size, output channels unchanged
3. **MaxPool2d(2)** - Max pooling with kernel size 2, reduces spatial size by half
4. **nn.Sequential(*layers)** - Sequential container that applies layers in order
5. **nn.Conv2d(in_c, out_c, 3, padding=1)** - 2D convolution: kernel size 3, padding 1 preserves spatial size
6. **ClassifierHead(3 * 16 * 16, 10)** - For example if input is 32x32x3. After 2 ConvBlocks (padding=1) size stays 32x32. MaxPool2d(2) reduces to 16x16. With 3 channels: $3 \times 16 \times 16 = 768$

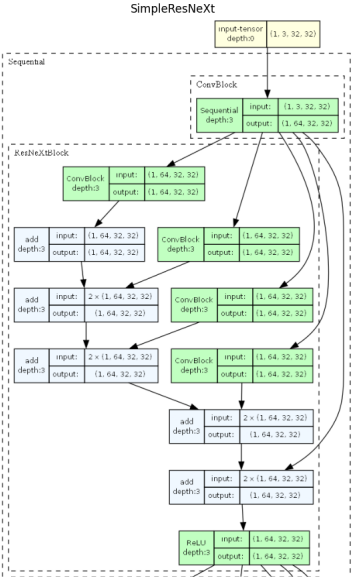
Part 2: ResNeXt and GoogLeNet

- **ResNeXtBlock:** Multiple ConvBlocks in parallel with skip connection
- **SimpleResNeXt:** ConvBlock + 2 ResNeXtBlocks + ClassifierHead
- **InceptionBlock:** 3 parallel ConvBlocks concatenated
- **SimpleGoogLeNet:** ConvBlock (stem) + 2 InceptionBlocks + 2 ClassifierHeads
- **GoogLeNetLoss:** Combines main and auxiliary classifier losses
- Both architectures explore different ways to increase model capacity

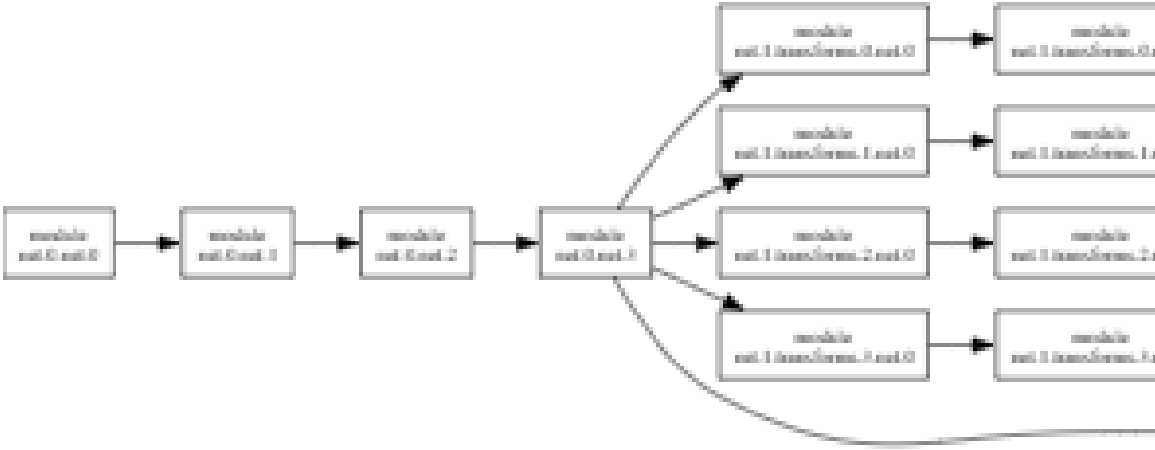
Part 2: SimpleResNeXt Architecture (1)

SimpleResNeXt

Part 2: SimpleResNeXt Architecture (2)



Part 2: SimpleResNeXt Architecture (2)



Part 2: Help Info

1. **nn.ModuleList** - List of modules that can be indexed like a regular list. Use for parallel paths
2. **torch.cat([b1(x), b3(x), b5(x)], dim=1)** - Concatenates tensors along channel dimension (dim=1)
3. **ResNeXtBlock:** Sum parallel ConvBlock outputs: $y = x + \sum_{i=1}^{cardinality} \text{ConvBlock}_i(x)$

Part 3: Transfer Learning

- **Goal:** Use a pretrained model instead of training from scratch
- **Idea:** Reuse learned features from a model trained on a large dataset
- **Example:** Pretrained ResNet18 + new classifier head
- In this lab we compare transfer learning with our own CNN architectures

Part 3: TransferResNet18 Architecture

- **Backbone:** pretrained ResNet18 from `torchvision.models`
- **Freeze parameters:** pretrained layers are not updated
- **Replace final layer:** `model.fc = nn.Linear(in_features, num_classes)`
- **Train only classifier:** simple transfer learning setup

Part 3: Help Info

1. **torchvision.models.resnet18(weights=...)** - Loads pretrained ResNet18
2. **param.requires_grad = False** - Freezes pretrained backbone parameters
3. **model.fc** - Final classification layer that can be replaced for a new task